

Runbook Setup — Ytasty Crousty G3

Sep 18, 2026 · rédigé par @Emrick

Vue d'ensemble

Ce runbook couvre les tâches Setup du projet Ytasty Crousty (S1, S5, S3, S4, et S2 repris en plus) : initialisation du dépôt, route de santé, conteneurisation, connexion à PostgreSQL, et les 6 modèles de données avec leurs relations. Il sert de mémo pour relancer l'environnement de dev et pour expliquer les choix techniques au reste de l'équipe.

S1 — Initialisation du projet

Gestion des dépendances avec `uv` plutôt que `pip/venv` classique, car toute l'équipe utilise cet outil.

```
uv init --app --no-readme --vcs none
uv add fastapi "uvicorn[standard]"
uv add sqlalchemy psycpg2-binary
```

Points à retenir :

- `requires-python` dans `pyproject.toml` doit rester `>=3.11` (le minimum du cahier des charges), pas la version locale exacte (`uv init` le détecte parfois trop haut).
- `.gitignore` doit ignorer `.venv/` (avec le point — c'est le dossier que crée `uv`, différent du `venv/` classique).
- Structure : `src/` contient le code applicatif (`main.py`, `db/`, `models/`), avec un fichier par modèle SQLAlchemy.

S5 — Route GET /health

Route de santé imposée par le contrat, testée en local puis en container.

```
@app.get("/health")
def health():
    return {"status": "ok"}
```

Réponse attendue : `{"status": "ok"}`, code HTTP 200. Vérifiée sur `http://127.0.0.1:8000/health` — pas `localhost`, qui résout parfois en IPv6 et échoue avec Docker Desktop sous Windows.

S3 — Docker & docker-compose

Dockerfile (copie les fichiers de dépendances avant le code pour profiter du cache de layers Docker) :

```
FROM python:3.12-slim
COPY --from=ghcr.io/astral-sh/uv:latest /uv /uvx /bin/
WORKDIR /app
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-cache
COPY . .
EXPOSE 8000
CMD ["sh", "-c", "/app/.venv/bin/uvicorn src.main:app --host 0.0.0.0 --port ${PORT:-8000}"]
```

docker-compose.yml (service `api` + service `db` Postgres, valeurs lues depuis `.env`) :

```
services:
  api:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: ${DATABASE_URL}
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      POSTGRES_DB: ${POSTGRES_DB}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data

volumes:
  pgdata:
```

Lancement : `docker compose up --build`. Le volume `pgdata` conserve les données entre redémarrages ; sans lui, tout serait perdu à chaque `docker compose down`.

S4 — Connexion SQLAlchemy + PostgreSQL

`src/db/database.py` :

```
import os
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = os.environ.get(
    "DATABASE_URL",
    "postgresql://ytastyg3:ytastyg3@127.0.0.1:5432/ytastyg3",
)

engine = create_engine(DATABASE_URL, pool_pre_ping=True)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Deux adresses possibles pour la même base Postgres :

Contexte	Hôte utilisé	Pourquoi
Via <code>docker compose up</code>	<code>db</code>	nom du service Postgres, résolu automatiquement dans le réseau Docker interne
API lancée hors Docker (<code>uv run uvicorn</code>)	<code>127.0.0.1</code>	le port 5432 du container <code>db</code> est mappé vers la machine hôte

Driver : `psycopg2-binary` — version précompilée de `psycopg2`, pas de compilation nécessaire sous Windows/Docker. `get_db()` ouvre une session par requête et la ferme via `yield / finally`, utilisé comme dépendance FastAPI (`Depends(get_db)`).

S2 — Modèles et relations

Un fichier par modèle dans `src/models/`, tous enregistrés via `src/models/__init__.py` (nécessaire pour que `Base.metadata.create_all` les voie). `relationship()` prend le nom de classe Python ; `ForeignKey()` prend le nom de table SQL.

Modèle	Table SQL	Colonnes clés	Relations
Restaurant	restaurants	name, city, address, is_open (défaut True), opening_hours, contact	users, produits, orders (1→N)
User	users	first_name, last_name, username, hashed_password, role, restaurant_id	restaurant (N→1)
Produit	produits	name, category, price (Numeric(6,2)), is_available, restaurant_id (nullable=False), ingredients (ARRAY(String))	restaurant (N→1), order_items (1→N)
Ingredient	ingredients	name (unique)	produits — relation N↔N via produit_ingredient remplacée par Produit.ingredients (ARRAY(String)) au Lot B, devenue obsolète
Order	orders	order_number (unique), created_at, total_price (Numeric(6,2)), status (Enum), pickup_mode (Enum), customer_name, customer_email, restaurant_id	restaurant (N→1), order_items (1→N)
OrderItem	order_items	quantity, prix_fige_commande (Numeric(4,2)) — prix figé au moment de la commande, product_id, order_id	produit (N→1), order (N→1)

Trois notions clés apprises pour ces modèles :

- **Enum** : `class Status(str, enum.Enum)` — double héritage pour que chaque valeur soit à la fois membre de l'Enum et vraie chaîne de caractères (comparaison directe avec du JSON).

- **Table d'association simple** (`produit_ingredient`) : objet `Table` brut, pas de classe, utilisé quand la relation N↔N n'a aucune donnée en plus des deux clés étrangères.
- **Objet d'association** (`OrderItem`) : vraie classe héritant de `Base` , utilisée dès qu'une relation N↔N porte des données en plus (ici `quantity` et le prix figé).

Variables d'environnement

`.env.example` (commité, sert de modèle à l'équipe) :

```
POSTGRES_USER=changeme
POSTGRES_PASSWORD=changeme
POSTGRES_DB=changeme
DATABASE_URL=postgresql://changeme:changeme@db:5432/changeme
```

`.env` contient les vraies valeurs locales et reste ignoré par Git (`.gitignore`). Docker Compose lit automatiquement un fichier nommé `.env` à côté de `docker-compose.yml` et substitue chaque `{{VARIABLE}}` — aucune configuration `env_file` : supplémentaire nécessaire.

Lifespan et création des tables

`src/main.py` :

```
from fastapi import FastAPI
from contextlib import asynccontextmanager
from src.db.database import Base, engine
import src.models

@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)
    yield

app = FastAPI(title="ytastycrousty-g3", lifespan=lifespan)

@app.get("/health")
def health():
    return {"status": "ok"}
```

`import src.models` est indispensable avant `create_all` : sans lui, aucune classe modèle n'a été chargée par Python, donc `Base.metadata` ne connaît aucune table à créer. `lifespan` remplace l'ancien `@app.on_event("startup")`, déprécié par FastAPI.

Commandes utiles

Commande	Effet
<code>uv sync</code>	installe les dépendances depuis <code>pyproject.toml</code> / <code>uv.lock</code>
<code>uv add <package></code>	ajoute une dépendance
<code>uv run uvicorn src.main:app --reload</code>	lance l'API en local, hors Docker
<code>docker compose up --build</code>	reconstruit et lance l'API + Postgres
<code>docker compose exec db psql -U ytastyg3 -d ytastyg3</code>	ouvre une session <code>psql</code> dans le container Postgres
<code>\dt</code> (dans <code>psql</code>)	liste les tables
<code>uv run python -c "import src.models; print('OK')"</code>	vérifie que tous les modèles se chargent sans erreur
<code>git checkout -b feature/<nom></code>	crée et bascule sur une nouvelle branche

A1-A7 — Authentification et utilisateurs

Tâches Auth/Users du cahier des charges, réalisées sur la branche `feature/auth`. Le modèle `User` existait déjà (construit pendant S2) ; ce lot ajoute le hashing du mot de passe, l'authentification JWT, le contrôle d'accès par rôle, et un CRUD complet pour Users (au-delà du minimum imposé par le contrat).

`src/core/security.py` :

```
from passlib.context import CryptContext
import jwt
import os
from datetime import datetime, timedelta

pwd_context = CryptContext(schemes=["bcrypt"])

def hashing_mdp(mdp: str) -> str:
```

```

return pwd_context.hash(mdp)

def verif_hash(mdp: str, mdp_hash: str) -> bool:
    return pwd_context.verify(mdp, mdp_hash)

SECRET_KEY = os.environ["SECRET_KEY"]
ALGORITHM = "HS256"

def creer_jwt(username: str, role: str) -> str:
    payload = {
        "sub": username,
        "role": role,
        "exp": datetime.utcnow() + timedelta(minutes=30),
    }
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def verif_jwt(token: str) -> dict:
    return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

```

Noms gardés en français pour ce fichier utilitaire interne. Incompatibilité rencontrée entre passlib 1.7.4 et bcrypt >=4.1 (MissingBackendError) — corrigée en pinnant la dépendance: `uv add "bcrypt<4.1,>=3.2.0"`.

`src/core/deps.py` (nouveau fichier, séparé de `security.py` qui reste pur/sans FastAPI)

:

```

from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer

from src.core.security import verif_jwt
from src.db.database import get_db

security = HTTPBearer(auto_error=False)

def get_current_user(credentials=Depends(security), db=Depends(get_db)):
    try:
        payload = verif_jwt(credentials.credentials)
    except Exception:
        raise HTTPException(401, "Token invalide ou expiré")
    user = db.query(User).filter_by(username=payload["sub"]).first()
    if user is None:

```



```

        raise HTTPException(401, "Token invalide ou expiré")
    return user

def require_role(*roles_autorises):
    def verificateur(user=Depends(get_current_user)):
        if user.role not in roles_autorises:
            raise HTTPException(403, "Rôle non autorisé")
        return user

    return verificateur

```

HTTPBearer plutôt que OAuth2PasswordBearer, incompatible avec le login JSON imposé par le contrat — ce choix couvre A7 (schéma Bearer OpenAPI) gratuitement. Nouvelle structure `src/routers/` (un fichier par domaine), démarrée avec `auth.py` : `POST /auth/login` (renvoie `{access_token, token_type: "bearer"}`) et `POST /users`.

Points à retenir :

- A3 : PyJWT choisi plutôt que python-jose (non maintenu depuis 2021, CVE connue). `SECRET_KEY` chargée sans fallback (`os.environ["SECRET_KEY"]`), fail-fast si absente.
- 401 générique sur `POST /auth/login`, sans distinguer user inconnu et mot de passe incorrect — anti brute-force.
- Le rôle est embarqué dans le payload JWT pour éviter une requête DB à chaque requête protégée, accepté avec l'expiration courte (30 min) comme limite de fraîcheur.
- A6 (`POST /users`) : réservée à `admin` via `require_role("admin")`, validation stricte (username 8-12 alphanumérique, password 12-64 avec chiffre/majuscule/caractère spécial), 400 si username déjà pris.
- CRUD complet ajouté pour Users (`GET /users`, `GET /users/{id}`, `PATCH /users/{id}`, `DELETE /users/{id}`), au-delà du POST imposé par le contrat — tous réservés à `admin`, `PATCH` en update partiel, pas de cas spécial pour l'auto-suppression d'un admin.
- Pour reset la table `users` en dev : `TRUNCATE TABLE users RESTART IDENTITY` puis redémarrer le conteneur `api` (`admin123` reseedé automatiquement par le lifespan).

Lot B — Restaurants et Produits

Lot Restaurants/Produits du cahier des charges, terminé après le lot Auth. Ajoute le CRUD complet pour Restaurants et Produits, l'autorisation par restaurant, et des seeds réalistes pour développer et démontrer sans données créées à la main.

Seeds (`src/db/seed.py` , sorti de `main.py` pour rester lisible et testable indépendamment) : appelés dans l'ordre `admin` → `restaurants` → `produits`, chaque étape dépendant des `restaurant_id` créés par la précédente. La session est ouverte **avant** le `try`, pas dedans, pour rester accessible dans le `finally`. Chaque fonction suit le même pattern : liste de dictionnaires, boucle, vérification d'existence, `Class(**dict)`, puis **un seul** `db.commit()` à la fin — idempotent (le seed peut tourner à chaque démarrage sans dupliquer) et atomique (un `KeyError` à mi-parcours ne laisse pas la base à moitié seedée).

```
def seed_restaurants(db: Session):
    restaurants = [
        {"name": "Aix", "city": "Aix-en-Provence"},
        {"name": "Lyon", "city": "Lyon"},
        {"name": "Paris", "city": "Paris"},
    ]
    for r in restaurants:
        if not db.query(Restaurant).filter_by(name=r["name"]).first():
            db.add(Restaurant(**r))

def seed_produits(db: Session):
    produits = [
        {"name": "Margherita", "restaurant_id": 1},
        {"name": "Margherita", "restaurant_id": 2},
    ]
    for p in produits:
        exists = db.query(Produit).filter_by(
            name=p["name"], restaurant_id=p["restaurant_id"]
        ).first()
        if not exists:
            db.add(Produit(**p))

def seed(db: Session):
    seed_admin(db)
    seed_restaurants(db)
    seed_produits(db)
    db.commit()
```

Comptes de test `staff` et `direction` : le seed des comptes (`staffaix1` / `stafflyon1` / `direction1`) a été retiré du code — créer un compte avec un rôle autre qu' `admin` passe désormais obligatoirement par `POST /users` avec le token `admin`, jamais par un mot de passe prévisible en base. Conséquence : ces comptes ne survivent pas à un changement de schéma, il faut les recréer à la main après chaque `docker compose down -v`.

Restaurants identifiés par `name` (pas par `id`) : clé stable pour vérifier l'existence au fil des redémarrages, et non modifiable via `PATCH` — donc jamais changée sous les pieds du seed. Produits seedés avec `restaurant_id` en dur (1 = Aix, 2 = Lyon, 3 = Paris), vérifiés par le couple `name` + `restaurant_id` plutôt que `name` seul — plusieurs restaurants peuvent avoir le même produit au menu.

Modèle Produit modifié : `ingredients` passe d'une relation plusieurs-à-plusieurs à un `ARRAY(String)` PostgreSQL — compromis assumé faute de temps avant la deadline pour refaire proprement tout le CRUD des ingrédients. `price` passe en `Numeric(6,2)` (`Numeric(4,2)` plafonnait trop bas). `restaurant_id` en `nullable=False` : un produit sans restaurant n'a pas de sens métier. Rappel : `create_all` ne modifie pas une table existante — après ce changement de schéma, il faut `docker compose down -v` pour repartir d'une base propre, sinon l'ancienne colonne reste et le nouveau code plante au démarrage.

Restaurants : `RestaurantUpdate` limité à `address` et `contact` — `name` doit rester stable (clé du seed), le reste se gère par des routes dédiées. `PATCH` renvoie toujours le `RestaurantOut` complet, même si le JSON du contrat ne montre qu'un exemple d'entrée partielle — le client doit voir l'état réel après modification, pas juste ce qu'il a envoyé. `availability` idempotent : rejouer le même appel (ex. après un timeout réseau) ne doit rien casser.

Produits : `ProductOut` renvoie `price` en `float` — `Numeric` n'est pas nativement sérialisable en JSON. Les filtres se combinent par étapes (`query.filter(...)` enchaînés), chaque filtre optionnel s'appliquant indépendamment sans construire une seule requête conditionnelle géante. `q` fait un `ilike` sur `name` OU `description` OU `array_to_string(ingredients)` via `or_` — il faut convertir `ingredients` en texte pour le comparer, maintenant que ce n'est plus une relation. Liste vide = `200 []`, jamais `404` : l'absence de résultat n'est pas une erreur. `ProductCreate` valide `price` (`gt=0`) et `min_length` sur les champs texte, pour rejeter les données invalides dès l'entrée. `ProductUpdate` exclut `restaurant_id` (un produit ne change pas de restaurant) et `is_available` (a sa propre route, cohérent avec `availability` côté Restaurants). `DELETE` refusé en `400` si le produit a des `order_items` : un produit déjà commandé ne doit pas disparaître de l'historique. `charger_produit` factorise la récupération + le `404` répétés dans plusieurs routes.

Autorisation par restaurant (`verifier_acces_restaurant` dans `deps.py`) : `admin` passe partout, `staff` seulement sur son propre restaurant, refus par défaut pour tout le reste — refuser par défaut plutôt qu'autoriser par défaut, pour qu'un rôle imprévu ne passe jamais par erreur.

```
def verifier_acces_restaurant(user, restaurant_id: int):
    if user.role == "admin":
        return
    if user.role == "staff" and user.restaurant_id == restaurant_id:
        return
    raise HTTPException(403, "Accès refusé pour ce restaurant")
```

Appelée dans le corps de la route, pas en `Depends` : le `restaurant_id` concerné n'est souvent connu qu'après avoir chargé la ressource (ex. via `charger_produit`), donc indisponible au moment où les `Depends` s'exécutent. Toujours vérifiée contre le `restaurant_id` du produit **en base**, jamais celui du body — sinon un `staff` pourrait mettre un `restaurant_id` différent dans sa requête pour contourner la restriction. `direction` reste en lecture seule.

Corrections du lot Auth faites pendant le lot B :

- Clé `access_token` manquante/mal nommée dans la réponse du login — corrigée, cassait l'intégration côté client qui attend exactement ce nom.
- `POST / PATCH /users` étaient censés vérifier que le restaurant existe et qu'un `staff` a bien un `restaurant_id` — en pratique cette vérification (`verifier_restaurant_user`) manquait encore ici et n'a été ajoutée que dans les corrections finales du lot C (voir « Lot C : corrections finales »).
- Règles de validation du mot de passe factorisées — étaient dupliquées entre création et mise à jour, risque de divergence.
- `HTTPBearer(auto_error=False)` — sans ce paramètre, FastAPI renvoyait un `403` générique quand aucun token n'était fourni, au lieu du `401` attendu par le contrat.

Pièges rencontrés :

- Blocs de seed copiés-collés qui divergent (le bloc Lyon créé sous le nom `"Aix"` par un copier-coller non corrigé).
- `Import from core` au lieu de `from src.core` (auto-import de l'IDE) : fonctionne en local, plante dans Docker où le répertoire de travail diffère.
- `str | None` sans `= None` reste un paramètre obligatoire — le type optionnel ne rend pas la valeur par défaut optionnelle.
- `KeyError` dans le seed quand les clés d'un dictionnaire ne correspondent pas aux paramètres attendus par la fonction.

Lot C — Commandes

Dernier lot fonctionnel du contrat, terminé après le lot B. Ajoute la prise de commande (prix figé, génération sécurisée du `order_number`, autorisations en lecture par restaurant) et referme les CRUD prévus : pas de suppression, seulement l'annulation.

Modèles corrigés : `Order` — `created_at` en `DateTime(timezone=True)` (sinon Postgres stocke sans fuseau et l'interprète implicitement en UTC, ambigu à l'affichage/comparaison) ; `total_price` en `Numeric(8,2)` (une commande cumule plusieurs produits, peut dépasser les bornes d'un prix unitaire) ; `status` avec `default=Status.pending` (une commande créée doit démarrer dans un état connu, jamais `NULL`) ; `restaurant_id` en `nullable=False` (même raison que `Produit` : une commande sans restaurant n'a pas de sens) ; client stocké à plat (`customer_name`, `customer_email`) plutôt qu'en table séparée, faute de compte client à relier. `OrderItem` — `prix_fige_commande` en `Numeric(6,2)` (aligné sur la précision du prix produit) ; `product_id` et `order_id` en `nullable=False` (une ligne sans commande ni produit n'a aucun sens).

Prix figé : le prix est copié dans la ligne de commande au moment de la commande, pour qu'un changement ultérieur de `Produit.price` ne modifie pas rétroactivement le montant de commandes déjà passées.

`order_number` : "YC-" + 8 caractères majuscules/chiffres tirés avec le module `secrets` (jamais `random`) — `GET /orders/{order_number}` est public et expose nom et email du client, le numéro fait donc office de clé d'accès ; un générateur non cryptographique ou un numéro séquentiel serait devinable ou énumérable. Boucle anti-collision en plus du `unique=True` en base, comme filet de sécurité final.

```
import secrets
import string

def generer_order_number(db: Session) -> str:
    alphabet = string.ascii_uppercase + string.digits
    while True:
        candidate = "YC-" + "".join(secrets.choice(alphabet) for _ in
range(8))
        if not db.query(Order).filter_by(order_number=candidate).first():
            return candidate
```

`POST /orders` : le prix est relu en base, jamais celui envoyé par le client — un prix client serait falsifiable. Le total est calculé en `Decimal` (exact au centime, pas d'erreur d'arrondi binaire), converti en `float` seulement à la sérialisation JSON. Refus en `400` si restaurant inexistant ou fermé, produit inexistant, d'un autre restaurant, ou indisponible — ce sont

des règles métier sur des identifiants par ailleurs valides, pas des erreurs de validation de schéma. Quantité `<= 0`, liste d'items vide ou `pickup_mode` invalide en 422 via Pydantic — erreurs de forme des données d'entrée, gérées au niveau validation plutôt qu'en logique métier. Tout est validé avant un seul commit, les lignes enregistrées via la relation (`order_items=lignes`) — même logique d'atomicité que les seeds : une commande partiellement enregistrée serait pire qu'aucune commande.

commande_vers_sortie : traduit le modèle (client à plat, `order_items`) vers la forme du contrat (`customer` imbriqué, `items` avec `unit_price`) — le contrat impose une forme JSON différente du schéma de stockage, une fonction de traduction unique évite de dupliquer cette logique dans chaque route. `charger_commande` lève 404 — même factorisation que `charger_produit`, pour ne pas répéter la vérification dans plusieurs routes.

Autorisations : nouvelle fonction `verifier_lecture_restaurant` dans `deps.py` (`admin` et `direction` lisent tout, `staff` seulement son restaurant) pour `GET /restaurants/{id}/orders` — séparée de `verifier_acces_restaurant` car la lecture doit rester ouverte à `direction`, qui n'a pas de `restaurant_id` et supervise tout. `PATCH` status et `cancel` utilisent `verifier_acces_restaurant` (`direction` refusée en écriture) — `direction` reste en lecture seule partout ailleurs dans l'app, pas de raison de faire exception ici.

Statuts : `PATCH` status permissif — le contrat n'impose aucune machine à états précise entre les statuts intermédiaires, donc pas de logique de transition inutile à maintenir. Exception pour les statuts finaux `collected` et `cancelled` (400 si on tente d'en sortir) — ce sont des états terminaux, en sortir romprait l'intégrité de l'historique. `cancel` idempotent (rejouer l'appel après un timeout réseau ne doit rien casser), refusé en 400 si la commande est déjà `collected` — on n'annule pas une commande déjà remise au client.

Pas de DELETE sur les commandes : une commande est une trace comptable, l'annulation en tient lieu — même logique que le refus de supprimer un produit déjà commandé (Lot B) : préserver l'intégrité de l'historique plutôt que faire disparaître des données.

Finitions

POST /restaurants (bonus, réservée à `admin`) corrigée : la vérification de doublon cherchait par erreur dans la table `users` au lieu de `restaurants.name` — elle ne détectait donc jamais de doublon réel, deux restaurants pouvaient être créés avec le même nom et casser la clé utilisée par le seed. Fonction renommée `create_restaurant`. `is_open` par défaut à `True` — un restaurant nouvellement créé doit être visible/utilisable immédiatement, pas caché par défaut.

Tags Swagger par router : Auth & Users, Restaurants, Produits, Commandes, Santé — sans tags, toutes les routes apparaissent à plat dans une seule liste dans `/docs`, illisible une fois tous les lots réunis ; les tags les regroupent par domaine.

État du projet

Toutes les routes du contrat sont en place et testées en local avec Docker. L'API n'est **pas encore déployée**.

Reste à faire :

- **T3** — déploiement HTTPS avec une base de production neuve
- **T2** — tags Swagger
- **T8** — vérification des secrets
- **T7** — tests sur l'URL publique
- **T1**

Lot C : corrections finales

Dernières corrections apportées après les premiers tests bout-en-bout du lot C, avant de considérer le contrat comme complet.

500 sur POST /orders : venait de la colonne `status` restée `NULL` — le `default=Status.pending` déclaré dans le modèle `Order` n'avait en réalité jamais été intégré au code. Corrigé à deux niveaux : dans le modèle lui-même, et en passant `status=Status.pending` explicitement à la création de la commande — le default du modèle protège tout insert qui l'oublierait ailleurs, le passer explicitement au point de création documente l'intention plutôt que de dépendre uniquement d'un comportement implicite de SQLAlchemy. `DateTime(timezone=True)` et `Numeric(8,2)` réellement appliqués en base via un `docker compose down -v` — rappel : `create_all` ne modifie pas une table existante (déjà noté au lot B), donc sans ce reset les anciennes commandes de test avec un schéma différent seraient restées en base.

Vérification du restaurant dans POST / PATCH /users : la règle « staff sans restaurant » et la vérification de l'existence du restaurant (`verifier_restaurant_user`) manquaient encore à ce stade malgré ce qu'indiquait le lot B — ajoutées pour de bon ici. Le `PATCH` vérifie l'état final (la valeur envoyée dans la requête, sinon la valeur actuelle déjà en base) avant toute modification — un `PATCH` partiel ne doit pas valider seulement les champs présents dans le body, sinon un `staff` déjà incohérent pourrait être mis à jour sur un autre champ sans jamais corriger l'incohérence existante.

Piège : le test « restaurant inexistant » était passé par accident — c'est en réalité l'erreur « username déjà pris » qui remontait en premier et masquait l'absence de la vraie vérification, donnant un faux sentiment que ce cas était couvert.

Swagger

Documentation Swagger/OpenAPI soignée en fin de projet pour livrer une API auto-documentée, plutôt qu'un ajout cosmétique de dernière minute.

Description de l'app : mode d'emploi (comment s'authentifier, où trouver le token) directement dans la description globale de FastAPI — un correcteur qui découvre l'API par `/docs` doit pouvoir s'en servir sans documentation externe.

openapi_tags : description par section, en plus des tags déjà posés sur chaque router depuis le lot B — les tags seuls groupent les routes visuellement, une description par tag explique en plus le rôle de chaque groupe directement dans l'UI Swagger.

persistAuthorization : le token saisi dans Swagger reste actif entre les appels — sans ça, il faut recoller le token à chaque requête testée dans l'UI, source d'oublis pendant une correction en direct.

summary + docstring sur chaque route : le `summary` donne le titre court affiché dans la liste des routes, la docstring le détail affiché une fois la route dépliée — deux lectures différentes (survol vs détail).

Codes d'erreur documentés via `src/core/docs.py` (`ErreurOut` , `ERR_400` / `401` / `403` / `404` et des combinaisons prêtes à l'emploi `REPONSES_ECRITURE` , `REPONSES_CREATION` , `REPONSES_LECTURE` , `REPONSES_LECTURE_PROTEGEE`), choisies route par route pour ne documenter que les codes réellement possibles sur cette route — un Swagger qui liste `401` / `403` / `404` sur une route publique en lecture serait trompeur ; les combinaisons évitent de redéfinir la liste à la main sur chaque route.

TokenOut et HealthOut ajoutés comme schémas de sortie — avant ça, `POST /auth/login` et `GET /health` n'avaient pas de schéma de réponse typé dans Swagger, leur forme JSON n'était visible qu'en lisant le code.

Exemples pré-remplis (`example=`) volontairement non faits.

Exposition et hébergement

Render + Neon étudiés puis abandonnés : le contrat impose ngrok, donc pas de réel choix côté hébergement une fois cette contrainte lue — à noter pour que l'équipe ne reparte pas sur cette piste plus tard en pensant qu'elle n'a pas été explorée.

Exposition : l'API et PostgreSQL tournent en local avec Docker (comme décrit au lot Setup), exposés en HTTPS par `ngrok http 8000` — HTTPS imposé par le contrat, et ngrok le fournit gratuitement sans certificat à gérer.

Limites du plan gratuit ngrok : 100 nouvelles connexions TCP par minute, 20 000 requêtes par mois — elles ont directement contraint le script de tests (voir Tests automatisés ci-dessous) et signifient que le PC, Docker et ngrok doivent rester allumés pendant toute la correction : pas d'hébergement permanent tant que T3 n'est pas fait.

Dockerfile rendu compatible avec une variable `PORT` (forme `sh -c` avec `${PORT:-8000}`, voir S3) — la forme `exec` précédente (`CMD [ ... ]`) ne permet pas la substitution de variables d'environnement, nécessaire pour un futur déploiement où le port est imposé par la plateforme plutôt que fixé en dur.

`pool_pre_ping` ajouté au moteur SQLAlchemy (voir S4) — une connexion PostgreSQL gardée ouverte peut être coupée silencieusement (proxy, idle timeout) sans que SQLAlchemy s'en aperçoive ; `pool_pre_ping` teste la connexion avant de la réutiliser et en ouvre une nouvelle si besoin, plutôt que de renvoyer une erreur au premier vrai `query`.

Dépendance inutile `"router"` retirée de `pyproject.toml` — jamais utilisée dans le code, gardée par erreur, alourdissait `uv sync` et la surface d'audit de sécurité pour rien.

Tests automatisés

`tests/test_contrat.py` : 135 scénarios issus de la fiche de tests fournie, écrit avec la seule bibliothèque standard (`urllib`) — éviter une dépendance de test supplémentaire (`requests` , `httpx`) sur un projet où l'installation doit rester simple pour la correction.

Données de test uniques par exécution (suffixe aléatoire) — rejouer la suite plusieurs fois sur la même base ne doit pas se heurter aux contraintes `unique=True` (username, name de restaurant, etc.) laissées par l'exécution précédente.

`try / finally` pour restaurer le restaurant modifié et supprimer les comptes de test créés pendant la suite — la suite doit pouvoir se rejouer indéfiniment sans accumuler de données ni laisser la base dans un état différent d'avant son passage.

Nouvelles tentatives en cas de coupure réseau — les appels passent par ngrok en HTTPS sur une connexion suffisamment instable pour qu'une coupure isolée ne doive pas faire échouer toute la suite.

Pause de 0,7 s entre requêtes via ngrok — `urllib` ouvre une connexion TCP par requête (pas de connection pooling), et sans cette pause la suite dépassait la limite de 100 nouvelles connexions par minute du plan gratuit ngrok, provoquant des erreurs SSL `UNEXPECTED_EOF` incompréhensibles sans creuser jusqu'à cette limite.

Résultat final : 135/135 en local et via ngrok.

Bilan

Toutes les routes du contrat sont implémentées et testées.

Principales leçons du projet :

- Tout valider avant d'écrire en base — un seul `commit()` à la fin plutôt que plusieurs écritures partielles.
- Vérifier l'autorisation contre la base, jamais contre le body de la requête — un client ne doit jamais pouvoir influencer la vérification de ses propres droits.
- Refus par défaut plutôt qu'autorisation par défaut, pour qu'un cas imprévu (rôle, statut) ne passe jamais par erreur.
- Idempotence des `PATCH` et des annulations — rejouer un appel après un timeout réseau ne doit rien casser.
- Éviter le copier-coller qui fait diverger le code (le bloc de seed Lyon nommé « Aix », entre autres) — préférer une fonction ou une boucle sur des données.
- Lire les logs du conteneur face à une `500` plutôt que de deviner — la vraie cause (colonne `NULL`, mauvaise table interrogée) n'était jamais visible depuis la réponse HTTP seule.
- Séparer bugs applicatifs et problèmes réseau en testant d'abord en local, avant d'accuser ngrok ou la connexion à chaque échec de test.